

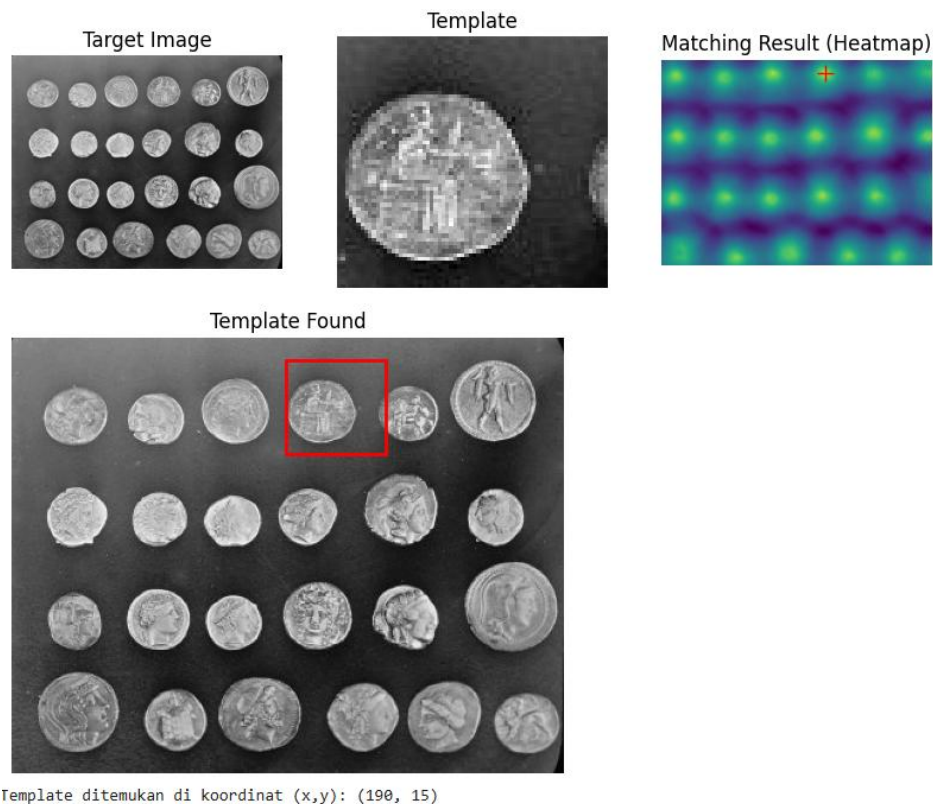


POLITEKNIK NEGERI SEMARANG
JURUSAN TEKNIK ELEKTRO
PROGRAM STUDI STR TEKNOLOGI REKAYASA KOMPUTER

JOBSHEET PRAKTIKUM

PENGOLAHAN CITRA

JOBSHEET 05: MENGUKUR KEMIRIPAN CITRA DAN PENERAPANNYA DALAM PENGENALAN POLA



Dosen Pengampu:
Ir. Prayitno, S.ST., M.T., Ph.D.

Nama Mahasiswa: _____

NIM: _____

Kelompok: _____



Semester Genap 2025

A. Tujuan Instruksional Khusus

Mahasiswa dapat menguraikan konsep segmentasi citra dengan indikator:

1. Menjelaskan konsep kemiripan citra dan perbedaannya dengan ketidakmiripan (jarak).
2. Mengidentifikasi dan menjelaskan berbagai ukuran kemiripan atau jarak antar citra/fitur (misalnya, Jarak Euclidean, Jarak Manhattan, Cosine Similarity, Structural Similarity Index/SSIM).
3. Menghitung ukuran kemiripan/jarak antara fitur citra sederhana atau patch citra.
4. Menjelaskan bagaimana ukuran kemiripan digunakan dalam tugas pengenalan pola seperti pencocokan template (template matching) dan pencarian citra berbasis konten (Content-Based Image Retrieval/CBIR).
5. Mengimplementasikan perhitungan kemiripan dasar dan aplikasi pengenalan pola sederhana menggunakan Python dan pustaka yang relevan (scikit-image, numpy, scipy).

B. Dasar Teori

1. Konsep Kemiripan Citra

Kemiripan citra (image similarity) adalah ukuran kuantitatif seberapa mirip dua citra secara visual atau berdasarkan fitur-fitur tertentu yang diekstraksi darinya. Konsep ini fundamental dalam banyak tugas visi komputer dan pengenalan pola. Seringkali, sebelum mengukur kemiripan, citra direpresentasikan dalam bentuk vektor fitur (feature vector) yang menangkap informasi penting seperti warna, tekstur, atau bentuk. Ukuran kemiripan biasanya menghasilkan nilai tinggi untuk citra yang mirip dan nilai rendah untuk citra yang berbeda. Sebaliknya, ukuran ketidakmiripan atau jarak (distance measure) menghasilkan nilai rendah (mendekati nol) untuk citra yang identik dan nilai tinggi untuk citra yang sangat berbeda.

2. Ukuran Kemiripan/Jarak

Terdapat berbagai cara untuk mengukur kemiripan atau jarak antara dua citra atau representasi fiturnya:

- **Ukuran Berbasis Pikel:**

- **Jarak Euclidean (L_2):** Menghitung akar dari jumlah kuadrat perbedaan nilai piksel pada posisi yang bersesuaian. Jika I_1 dan I_2 adalah dua citra (atau patch) berukuran $M \times N$, jarak Euclidean dihitung sebagai:

$$L_2(I_1, I_2) = \sqrt{\sum_{i=1}^M \sum_{j=1}^N (I_1(i, j) - I_2(i, j))^2}$$

Metrik ini sensitif terhadap perubahan pencahayaan, translasi, dan rotasi.

- **Jarak Manhattan (L_1):** Menghitung jumlah dari nilai absolut perbedaan nilai piksel pada posisi yang bersesuaian:

$$L_1(I_1, I_2) = \sum_{i=1}^M \sum_{j=1}^N |I_1(i, j) - I_2(i, j)|$$

Seperti L_2 , metrik ini juga sensitif terhadap pergeseran spasial dan perubahan iluminasi.

- **Sum of Squared Differences (SSD):** Ukuran ini menghitung jumlah dari kuadrat perbedaan intensitas antara piksel-piksel yang bersesuaian pada dua citra (atau patch) I_1 dan I_2 :

$$SSD(I_1, I_2) = \sum_{i=1}^M \sum_{j=1}^N (I_1(i, j) - I_2(i, j))^2$$

SSD adalah ukuran ketidakmiripan (dissimilarity), di mana nilai yang lebih rendah menunjukkan kemiripan yang lebih tinggi. Nilai SSD sama dengan nol jika dan hanya jika I_1 dan I_2 identik. Terlihat dari rumus bahwa SSD adalah kuadrat dari Jarak Euclidean. SSD sering digunakan dalam algoritma pencocokan template (template matching), di mana tujuannya adalah mencari lokasi dengan nilai SSD minimum antara template dan bagian dari citra target. Seperti L_1 dan L_2 , SSD juga sensitif terhadap perubahan iluminasi, translasi, dan rotasi.

- **Ukuran Berbasis Fitur:**

- **Cosine Similarity:** Mengukur kosinus sudut antara dua vektor fitur (misalnya, histogram warna, vektor fitur dari deep learning). Nilai berkisar antara -1 (berlawanan) hingga 1 (identik), dengan 0 menunjukkan ortogonalitas (tidak ada kemiripan). Untuk vektor fitur A dan B:

$$Similarity_{cosine}(A, B) = \frac{A \cdot B}{||A|| ||B||} = \frac{\sum_{i=1}^n A_i B_i}{\sqrt{\sum_{i=1}^n A_i^2} \sqrt{\sum_{i=1}^n B_i^2}}$$

Ukuran ini tidak sensitif terhadap magnitudo vektor, hanya orientasinya.

- **Ukuran Struktural/Perseptual:**

- **Structural Similarity Index (SSIM):** Ukuran kemiripan yang dirancang agar lebih sesuai dengan persepsi visual manusia dibandingkan metrik berbasis piksel sederhana seperti MSE atau PSNR. SSIM membandingkan tiga komponen: luminansi (luminance), kontras (contrast), dan struktur (structure) antara dua citra. Nilai SSIM berkisar antara -1 hingga 1, di mana 1 menunjukkan kesamaan struktural yang sempurna. SSIM dihitung secara lokal pada jendela (window) citra dan seringkali dirata-ratakan untuk mendapatkan skor global.
- Ukuran seperti Jarak Euclidean atau SSD terkadang memberikan nilai jarak yang rendah (menandakan mirip) padahal secara visual citra terlihat berbeda oleh manusia, atau sebaliknya. Ini karena ukuran tersebut hanya melihat perbedaan nilai piksel absolut tanpa mempertimbangkan struktur atau pola dalam citra. SSIM dikembangkan untuk mengatasi kelemahan ini dengan mencoba meniru cara sistem visual manusia (Human Visual System - HVS) memandang kemiripan. HVS sangat baik dalam mengenali *struktur* dalam sebuah objek atau pemandangan.
- SSIM tidak membandingkan piksel satu per satu secara global. Sebaliknya, SSIM bekerja pada bagian-bagian kecil dari citra (disebut *window* atau *patch*) dan membandingkan tiga aspek penting antara patch di citra pertama (x) dan patch di citra kedua (y) pada lokasi yang sama:
 - **Luminansi (Kecerahan):** Membandingkan tingkat kecerahan rata-rata pada patch x dan patch y. Apakah kedua patch sama-sama terang atau sama-sama gelap? Ini diukur menggunakan nilai rata-rata (mean) piksel di dalam patch.
 - **Kontras (Contrast):** Membandingkan rentang kecerahan (variasi dari rata-rata) pada patch x dan patch y. Apakah keduanya memiliki tingkat detail

atau variasi intensitas yang sama? Ini diukur menggunakan standar deviasi piksel di dalam patch.

- **Struktur (Structure):** Setelah memperhitungkan perbedaan luminansi dan kontras, SSIM membandingkan bagaimana pola piksel tersusun di dalam patch x dan y . Apakah pola garis, tepi, atau teksturnya mirip? Ini seringkali diukur menggunakan korelasi antara kedua patch setelah normalisasi.
- Hasil perbandingan ketiga komponen (luminansi, kontras, struktur) ini kemudian digabungkan (biasanya dikalikan) untuk menghasilkan skor SSIM *lokal* untuk window/patch tersebut. Skor ini berkisar antara -1 hingga 1.
- Untuk mendapatkan satu skor SSIM untuk keseluruhan citra, skor SSIM lokal dari semua window/patch yang dianalisis kemudian dirata-ratakan. Skor SSIM global yang mendekati 1 menunjukkan kedua citra sangat mirip secara struktural menurut persepsi manusia, sementara skor mendekati -1 menunjukkan sangat tidak mirip. Skor 0 menunjukkan tidak ada kemiripan struktural.

3. Penerapan dalam Pengenalan Pola

Ukuran kemiripan adalah kunci dalam berbagai aplikasi pengenalan pola:

- **Pencocokan Template (Template Matching):** Teknik untuk menemukan lokasi suatu citra kecil (template) di dalam citra yang lebih besar. Ini dilakukan dengan menggeser template ke seluruh area citra besar dan menghitung ukuran kemiripan (atau ketidakmiripan) di setiap posisi. Posisi dengan kemiripan tertinggi (atau ketidakmiripan terendah) dianggap sebagai lokasi template. Metode umum menggunakan Normalized Cross-Correlation (NCC) atau Sum of Squared Differences (SSD).
- **Pencarian Citra Berbasis Konten (Content-Based Image Retrieval - CBIR):** Sistem CBIR bertujuan untuk mencari dan mengambil citra dari database besar yang secara visual mirip dengan citra query yang diberikan pengguna. Prosesnya melibatkan ekstraksi fitur visual (seperti warna, tekstur, bentuk) dari citra query dan semua citra dalam database, kemudian menghitung kemiripan antara vektor fitur query dengan vektor fitur setiap citra di database menggunakan ukuran kemiripan yang sesuai (misalnya, Jarak Euclidean, Cosine Similarity). Citra dengan kemiripan tertinggi akan ditampilkan sebagai hasil pencarian.
- **Klasifikasi Objek (Nearest Neighbor):** Dalam algoritma klasifikasi seperti k-Nearest Neighbors (k-NN), sebuah objek baru diklasifikasikan berdasarkan kelas mayoritas dari 'tetangga' terdekatnya dalam ruang fitur. Kedekatan (kemiripan) diukur menggunakan metrik jarak/kemiripan yang sesuai.

C. Alat dan Bahan

Untuk dapat melaksanakan praktik jobsheet ini diperlukan:

- Komputer dengan Python terinstal
- Pustaka Python: scikit-image, matplotlib, numpy
- Contoh gambar dari scikit-image

D. Langkah Kerja Praktik

Praktikum 1. Menghitung Jarak Berbasis Piksel (Euclidean & Manhattan)

Praktikum ini bertujuan menghitung dan membandingkan jarak Euclidean (L2) dan Manhattan (L1) antara dua patch citra sederhana.

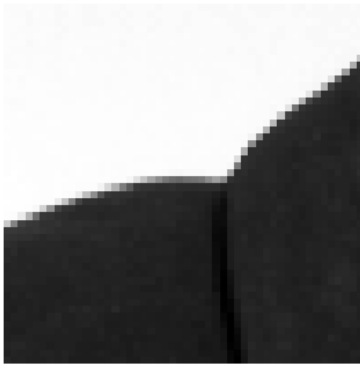
Kode Praktikum:

```
01: import numpy as np
02: from skimage import data, img_as_float
03: from scipy.spatial import distance
04: import matplotlib.pyplot as plt
05:
06: # 1. Buat dua patch citra sederhana atau ambil dari citra asli
07: image = img_as_float(data.camera())
08: patch1 = image[100:150, 100:150]
09: patch2 = image[100:150, 300:350] # Patch dari lokasi berbeda
10: patch3 = patch1 + 0.1 # Patch1 dengan sedikit noise/perubahan intensitas
11: patch3 = np.clip(patch3, 0, 1) # Pastikan nilai tetap di [0, 1]
12:
13: # 2. Flatten patch menjadi vektor 1D
14: vec1 = patch1.flatten()
15: vec2 = patch2.flatten()
16: vec3 = patch3.flatten()
17:
18: # 3. Hitung Jarak Euclidean (L2)
19: dist_l2_l2 = distance.euclidean(vec1, vec2)
20: dist_l2_l3 = distance.euclidean(vec1, vec3)
21:
22: # 4. Hitung Jarak Manhattan (L1 - disebut 'cityblock' di scipy)
23: dist_l1_l2 = distance.cityblock(vec1, vec2)
24: dist_l1_l3 = distance.cityblock(vec1, vec3)
25:
26: # 5. Tampilkan hasil dan patch
27: print(f"Jarak Euclidean antara Patch 1 dan Patch 2: {dist_l2_l2:.4f}")
28: print(f"Jarak Euclidean antara Patch 1 dan Patch 3: {dist_l2_l3:.4f}")
29: print(f"Jarak Manhattan antara Patch 1 dan Patch 2: {dist_l1_l2:.4f}")
30: print(f"Jarak Manhattan antara Patch 1 dan Patch 3: {dist_l1_l3:.4f}")
31:
32: fig, axes = plt.subplots(1, 3, figsize=(9, 3))
33: axes[0].imshow(patch1, cmap='gray')
34: axes[0].set_title('Patch 1')
35: axes[0].axis('off')
36: axes[1].imshow(patch2, cmap='gray')
37: axes[1].set_title('Patch 2')
38: axes[1].axis('off')
39: axes[2].imshow(patch3, cmap='gray')
40: axes[2].set_title('Patch 3 (Mirip Patch 1)')
41: axes[2].axis('off')
42: plt.tight_layout()
43: plt.show()
```

Hasil Praktikum.

```
Jarak Euclidean antara Patch 1 dan Patch 2: 24.7650  
Jarak Euclidean antara Patch 1 dan Patch 3: 5.0000  
Jarak Manhattan antara Patch 1 dan Patch 2: 936.4392  
Jarak Manhattan antara Patch 1 dan Patch 3: 250.0000
```

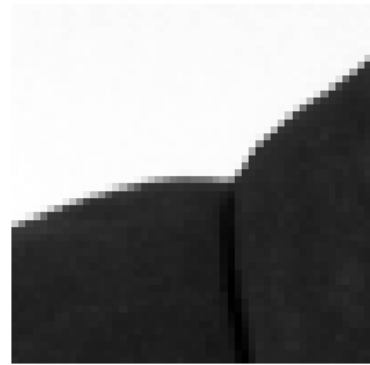
Patch 1



Patch 2



Patch 3 (Mirip Patch 1)



Amati nilai jarak yang dihasilkan. Bandingkan jarak antara patch1 & patch2 (berbeda lokasi) dengan patch1 & patch3 (mirip). Apakah nilai jarak sesuai dengan ekspektasi visual (jarak lebih kecil untuk patch yang lebih mirip)? Bandingkan tren antara jarak L1 dan L2.

Praktikum 2. Menghitung Cosine Similarity antara Histogram Warna

Praktikum ini menghitung kemiripan antara dua citra berwarna berdasarkan histogram warnanya menggunakan Cosine Similarity.

Kode Praktikum:

```
01: import numpy as np
02: import matplotlib.pyplot as plt
03: from skimage import data, img_as_ubyte, io, color
04: from scipy.spatial import distance
05:
06: # Fungsi untuk menghitung histogram RGB gabungan
07: def calculate_rgb_histogram(image, bins=16):
08:     img_uint8 = img_as_ubyte(image)
09:     hist_r, _ = np.histogram(img_uint8[:, :, 0].ravel(), bins=bins,
range=(0, 256))
10:     hist_g, _ = np.histogram(img_uint8[:, :, 1].ravel(), bins=bins,
range=(0, 256))
11:     hist_b, _ = np.histogram(img_uint8[:, :, 2].ravel(), bins=bins,
range=(0, 256))
12:     # Gabungkan histogram R, G, B menjadi satu vektor fitur
13:     hist_combined = np.concatenate((hist_r, hist_g, hist_b))
14:     # Normalisasi (opsional, tapi sering dilakukan)
15:     hist_combined = hist_combined.astype(float) / np.sum(hist_combined)
16:     return hist_combined
17:
18: # 1. Muat dua citra berwarna
19: try:
20:     # Gunakan citra berbeda dari skimage atau file Anda
21:     image1 = data.astronaut()
22:     image2 = data.coffee() # Citra yang berbeda
23:     image3 = data.astronaut() # Citra yang sama (untuk perbandingan)
24:     image4 = image1[:, :2, ::2, ::2, :] # Versi downsampled dari image1
25: except Exception as e:
26:     print(f"Gagal memuat data skimage: {e}. Membuat citra dummy.")
27:     image1 = np.random.rand(100, 100, 3)
28:     image2 = np.random.rand(100, 100, 3) * 0.5
29:     image3 = image1.copy()
30:     image4 = image1[:, :2, ::2, ::2, :]
31:
32:
33: # 2. Hitung histogram untuk setiap citra
34: hist1 = calculate_rgb_histogram(image1)
35: hist2 = calculate_rgb_histogram(image2)
36: hist3 = calculate_rgb_histogram(image3)
37: hist4 = calculate_rgb_histogram(image4)
38:
39: # 3. Hitung Cosine Similarity (1 - Cosine Distance)
40: # scipy.spatial.distance.cosine menghitung jarak (1 - similarity)
41: sim_12 = 1 - distance.cosine(hist1, hist2)
42: sim_13 = 1 - distance.cosine(hist1, hist3)
43: sim_14 = 1 - distance.cosine(hist1, hist4)
44:
```

```
45: # 4. Tampilkan hasil
46: print(f"Cosine Similarity antara Image 1 (Astronaut) dan Image 2 (Coffee): {sim_12:.4f}")
47: print(f"Cosine Similarity antara Image 1 (Astronaut) dan Image 3 (Astronaut): {sim_13:.4f}")
48: print(f"Cosine Similarity antara Image 1 (Astronaut) dan Image 4 (Astronaut Downsampled): {sim_14:.4f}")
49:
50: # Visualisasi (opsional)
51: fig, axes = plt.subplots(1, 4, figsize=(12, 3))
52:     axes[0].imshow(image1);     axes[0].set_title('Image 1');
axes[0].axis('off')
53:     axes[1].imshow(image2);     axes[1].set_title('Image 2');
axes[1].axis('off')
54:     axes[2].imshow(image3);     axes[2].set_title('Image 3 (Same)');
axes[2].axis('off')
55:     axes[3].imshow(image4);     axes[3].set_title('Image 4 (Downsampled)');
axes[3].axis('off')
56: plt.show()
```

Hasil Praktikum

Cosine Similarity antara Image 1 (Astronaut) dan Image 2 (Coffee): 0.8156
Cosine Similarity antara Image 1 (Astronaut) dan Image 3 (Astronaut): 1.0000
Cosine Similarity antara Image 1 (Astronaut) dan Image 4 (Astronaut Downsampled): 1.0000



Amati nilai Cosine Similarity. Apakah nilai untuk citra yang identik (Image 1 & 3) mendekati 1? Bagaimana kemiripan antara citra yang berbeda (Image 1 & 2)? Bagaimana kemiripan antara citra asli dan versi downsampled-nya (Image 1 & 4) berdasarkan histogram?

Praktikum 3. Menghitung Structural Similarity Index (SSIM)

Praktikum ini mengukur kemiripan struktural antara dua citra menggunakan SSIM.

Kode Praktikum:

```
01: import numpy as np
02: import matplotlib.pyplot as plt
03: from skimage import data, img_as_float
04: from skimage.metrics import structural_similarity as ssim
05: from skimage.transform import resize
06: from skimage.util import random_noise
07:
08: # 1. Muat citra referensi
09: image_ref = img_as_float(data.camera())
10:
11: # 2. Buat beberapa versi citra yang 'terdistorsi'
12: # a) Citra yang sama (SSIM harus 1)
13: image_same = image_ref.copy()
14: # b) Citra dengan noise Gaussian
15: image_noisy = random_noise(image_ref, mode='gaussian', var=0.01)
16: # c) Citra dengan kontras berbeda (contoh: dikalikan skalar)
17: image_contrast = np.clip(image_ref * 0.8, 0, 1)
18: # d) Citra yang di-blur
19: from skimage.filters import gaussian
20: image_blurred = gaussian(image_ref, sigma=1.5, channel_axis=None) #
channel_axis=None jika grayscale
21:
22:
23: # 3. Hitung SSIM antara citra referensi dan citra terdistorsi
24: # Pastikan dimensi citra sama jika diperlukan
25: # SSIM memerlukan data_range
26: data_range = image_ref.max() - image_ref.min()
27:
28: ssim_same, _ = ssim(image_ref, image_same, data_range=data_range,
full=True)
29: ssim_noisy, diff_noisy = ssim(image_ref, image_noisy,
data_range=data_range, full=True)
30: ssim_contrast, diff_contrast = ssim(image_ref, image_contrast,
data_range=data_range, full=True)
31: ssim_blurred, diff_blurred = ssim(image_ref, image_blurred,
data_range=data_range, full=True)
32: # 'full=True' mengembalikan citra perbedaan SSIM juga
33:
34: # 4. Tampilkan hasil SSIM dan citra perbedaan (jika menarik)
35: print(f"SSIM (Ref vs Same): {ssim_same:.4f}")
36: print(f"SSIM (Ref vs Noisy): {ssim_noisy:.4f}")
37: print(f"SSIM (Ref vs Contrast): {ssim_contrast:.4f}")
38: print(f"SSIM (Ref vs Blurred): {ssim_blurred:.4f}")
39:
40: fig, axes = plt.subplots(2, 4, figsize=(12, 6))
41: ax = axes.ravel()
42:
```

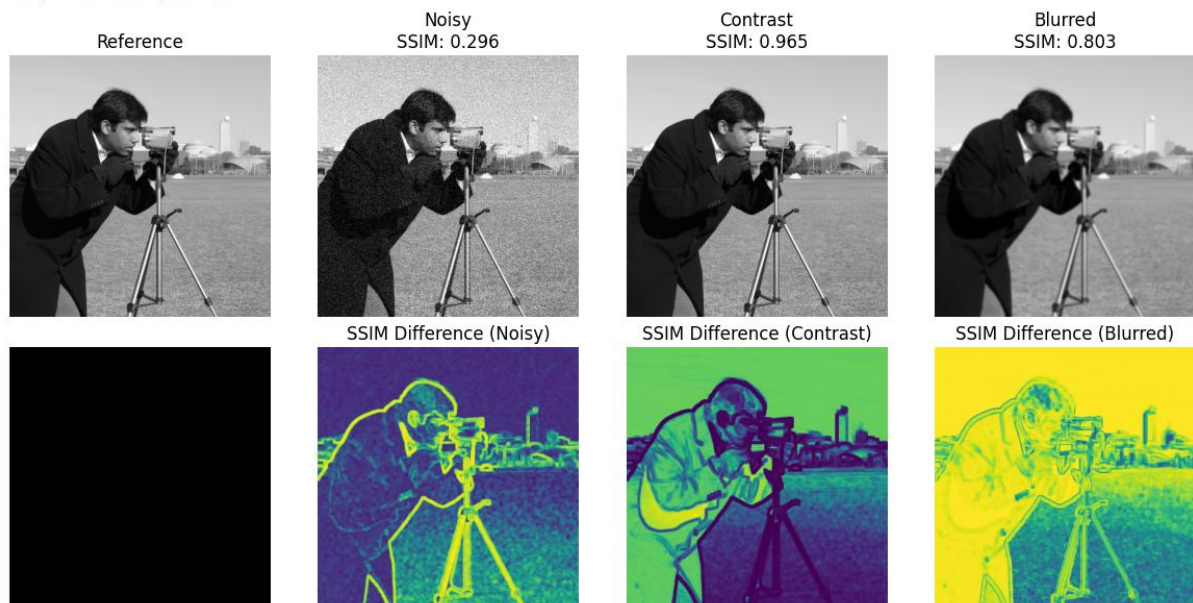
```

43: ax[0].imshow(image_ref, cmap='gray'); ax[0].set_title('Reference');
ax[0].axis('off')
44: ax[1].imshow(image_noisy, cmap='gray'); ax[1].set_title(f'Noisy\nSSIM:
{ssim_noisy:.3f}'); ax[1].axis('off')
45: ax[2].imshow(image_contrast, cmap='gray');
ax[2].set_title(f'Contrast\nSSIM: {ssim_contrast:.3f}'); ax[2].axis('off')
46: ax[3].imshow(image_blurred, cmap='gray');
ax[3].set_title(f'Blurred\nSSIM: {ssim_blurred:.3f}'); ax[3].axis('off')
47:
48: # Menampilkan peta perbedaan SSIM
49: ax[4].imshow(np.zeros_like(image_ref), cmap='gray');
ax[4].set_title(''); ax[4].axis('off') # Placeholder
50: ax[5].imshow(diff_noisy, cmap='viridis'); ax[5].set_title('SSIM
Difference (Noisy)'); ax[5].axis('off')
51: ax[6].imshow(diff_contrast, cmap='viridis'); ax[6].set_title('SSIM
Difference (Contrast)'); ax[6].axis('off')
52: ax[7].imshow(diff_blurred, cmap='viridis'); ax[7].set_title('SSIM
Difference (Blurred)'); ax[7].axis('off')
53:
54: plt.tight_layout()
55: plt.show()

```

Hasil Praktikum:

SSIM (Ref vs Same): 1.0000
SSIM (Ref vs Noisy): 0.2963
SSIM (Ref vs Contrast): 0.9651
SSIM (Ref vs Blurred): 0.8027



Amati nilai SSIM untuk setiap kasus. Apakah SSIM untuk citra yang sama bernilai 1? Bagaimana nilai SSIM untuk citra yang diberi noise, diubah kontrasnya, atau di-blur? Apakah nilai SSIM ini sesuai dengan persepsi visual Anda tentang kemiripan? Amati peta perbedaan SSIM, area mana yang menunjukkan ketidakmiripan struktural terbesar?

Praktikum 4. Penerapan Template Matching

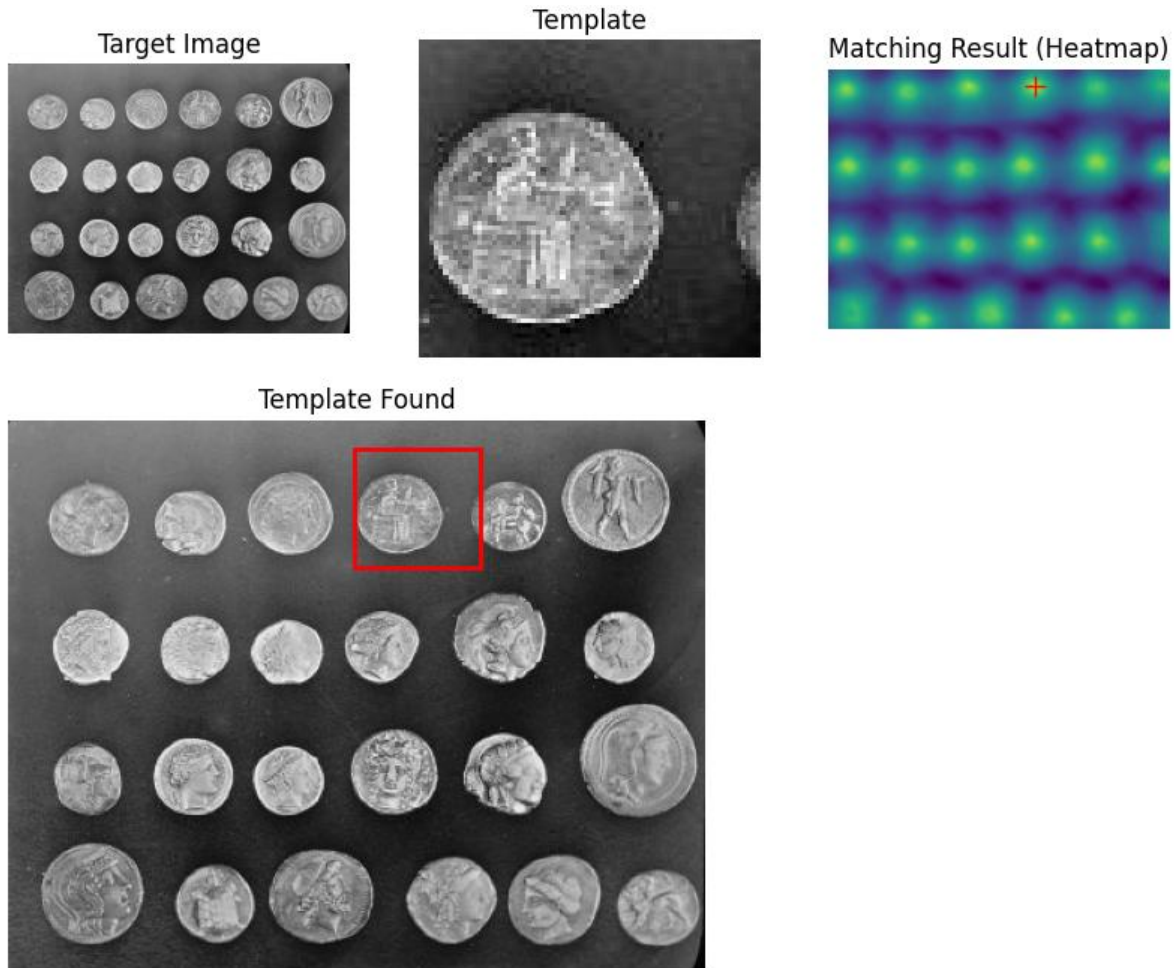
Praktikum ini menerapkan pencocokan template untuk menemukan lokasi objek kecil (template) dalam citra yang lebih besar.

Kode Praktikum:

```
01: import numpy as np
02: import matplotlib.pyplot as plt
03: from skimage import data
04: from skimage.feature import match_template
05:
06: # 1. Muat citra target dan buat/muat citra template
07: image = data.coins()
08: # Ambil salah satu koin sebagai template (sesuaikan koordinat jika perlu)
09: template = image[15:80, 190:260]
10:
11: # 2. Lakukan template matching menggunakan Normalized Cross-Correlation
    (metode default)
12: result = match_template(image, template)
13:
14: # 3. Temukan lokasi dengan skor matching tertinggi
15: # np.argmax menemukan indeks linear, unravel_index mengubahnya ke
    koordinat 2D
16: ij = np.unravel_index(np.argmax(result), result.shape)
17: x, y = ij[::-1] # Koordinat (x, y) dari sudut kiri atas template yang
    cocok
18:
19: # 4. Visualisasi hasil
20: fig, ax = plt.subplots(1, 3, figsize=(10, 4))
21:
22: ax[0].imshow(image, cmap='gray')
23: ax[0].set_title('Target Image')
24: ax[0].set_axis_off()
25:
26: ax[1].imshow(template, cmap='gray')
27: ax[1].set_title('Template')
28: ax[1].set_axis_off()
29:
30: # Menampilkan heatmap hasil matching (opsional)
31: ax[2].imshow(result, cmap='viridis')
32: ax[2].set_title('Matching Result (Heatmap)')
33: ax[2].set_axis_off()
34: # Tandai lokasi terbaik pada heatmap
35: ax[2].plot(x, y, 'r+', markersize=10) # y, x karena ij adalah (row, col)
36:
37: # Menampilkan kotak di lokasi terbaik pada citra asli (di plot terpisah
    agar lebih jelas)
38: fig2, ax_main = plt.subplots(figsize=(6, 6))
39: ax_main.imshow(image, cmap='gray')
40: ax_main.set_title('Template Found')
41: ax_main.set_axis_off()
42: h, w = template.shape
43: rect = plt.Rectangle((x, y), w, h, edgecolor='r', facecolor='none', lw=2)
```

```
44: ax_main.add_patch(rect)
45:
46: plt.show()
47:
48: print(f"Template ditemukan di koordinat (x,y): ({x}, {y})")
```

Hasil Praktik



Template ditemukan di koordinat (x,y): (190, 15)

Amati heatmap hasil `match_template`. Apakah area paling terang (nilai tertinggi) sesuai dengan lokasi template di citra asli? Apakah metode ini berhasil menemukan koin yang dijadikan template? Apa keterbatasan metode ini (misalnya, jika template dirotasi atau ukurannya berbeda)?

Praktikum 5. Simulasi Content-Based Image Retrieval (CBIR) Sederhana

Praktikum ini mensimulasikan CBIR sederhana dengan mencari citra yang paling mirip dengan query berdasarkan kemiripan histogram warna.

Kode Praktikum:

```
01: import numpy as np
02: import matplotlib.pyplot as plt
03: from skimage import data, io, color, transform, img_as_float
04: from scipy.spatial import distance
05:
06: # Fungsi hitung histogram dari Praktikum 2
07: def calculate_rgb_histogram(image, bins=16):
08:     # Pastikan input float [0,1] dikonversi ke uint8 [0,255] untuk
    histogram
09:     if image.dtype == float:
10:         image = img_as_ubyte(image)
11:     hist_r, _ = np.histogram(image[:, :, 0].ravel(), bins=bins, range=(0,
    256))
12:     hist_g, _ = np.histogram(image[:, :, 1].ravel(), bins=bins, range=(0,
    256))
13:     hist_b, _ = np.histogram(image[:, :, 2].ravel(), bins=bins, range=(0,
    256))
14:     hist_combined = np.concatenate((hist_r, hist_g, hist_b))
15:     # Normalisasi L1 (sum to 1)
16:     hist_sum = np.sum(hist_combined)
17:     if hist_sum > 0:
18:         hist_combined = hist_combined.astype(float) / hist_sum
19:     else:
20:         hist_combined = hist_combined.astype(float) # Avoid division by
    zero
21:     return hist_combined
22:
23: # 1. Siapkan 'database' citra kecil dan citra query
24: # Gunakan beberapa gambar dari skimage.data atau gambar Anda sendiri
25: image_db_names = ["astronaut", "camera", "coffee", "coins", "chelsea"]
26: database_images = []
27: database_hists = []
28:
29: print("Memproses database citra...")
30: for name in image_db_names:
31:     try:
32:         img = getattr(data, name)()
33:         # Pastikan semua citra berwarna (RGB) dan ukuran sama (opsional,
    resize)
34:         if img.ndim == 2: # Jika grayscale, konversi ke RGB
35:             img = color.gray2rgb(img)
36:         # Resize agar ukuran fitur konsisten (misal 100x100)
37:         img_resized = transform.resize(img, (100, 100),
    anti_aliasing=True)
38:         database_images.append(img_resized)
39:         database_hists.append(calculate_rgb_histogram(img_resized))
40:     print(f"- {name} diproses.")
```

```
41:     except Exception as e:
42:         print(f"Error memproses {name}: {e}")
43:
44: # Pilih satu citra sebagai query (misal, 'chelsea' si kucing)
45: query_image_name = "chelsea"
46: query_index = image_db_names.index(query_image_name)
47: query_image = database_images[query_index]
48: query_hist = database_hists[query_index]
49:
50: # 2. Hitung jarak/kemiripan antara query dan semua citra di database
51: # Kita gunakan Cosine Distance (1 - Cosine Similarity)
52: distances = []
53: for i, hist in enumerate(database_hists):
54:     # Hindari membandingkan query dengan dirinya sendiri (jarak akan 0)
55:     # if i == query_index:
56:     #     distances.append(np.inf) # Beri jarak tak hingga
57:     # else:
58:     dist = distance.cosine(query_hist, hist)
59:     distances.append(dist)
60:
61: # 3. Urutkan citra berdasarkan jarak (dari terkecil ke terbesar)
62: # Dapatkan indeks urutan jarak
63: sorted_indices = np.argsort(distances)
64:
65: # 4. Tampilkan hasil: Query dan citra terurut berdasarkan kemiripan
66: num_results_to_show = len(database_images)
67: fig, axes = plt.subplots(1, num_results_to_show + 1, figsize=(15, 3))
68:
69: # Tampilkan query
70: axes[0].imshow(query_image)
71: axes[0].set_title(f"Query: {query_image_name}")
72: axes[0].axis('off')
73:
74: # Tampilkan hasil terurut
75: print("\nHasil Retrieval (semakin ke kanan, semakin tidak mirip):")
76: for i, idx in enumerate(sorted_indices):
77:     # Lewati query itu sendiri di hasil
78:     # if i == 0: continue # Indeks pertama pasti query itu sendiri dengan
    jarak 0
79:     img_rank = i + 1 # Rank dimulai dari 1
80:     ax = axes[img_rank] # Mulai dari plot ke-2
81:     ax.imshow(database_images[idx])
82:     ax.set_title(f"Rank    {img_rank}\n{image_db_names[idx]}\nDist:
    {distances[idx]:.3f}")
83:     ax.axis('off')
84:     print(f"Rank    {img_rank}:    {image_db_names[idx]}    (Distance:
    {distances[idx]:.3f}")
85:
86: # Matikan axes yang tidak terpakai jika num_results < len(db)
87: for j in range(num_results_to_show + 1, len(axes)):
88:     axes[j].axis('off')
89:
90: plt.tight_layout()
```

```
91: plt.show()
```

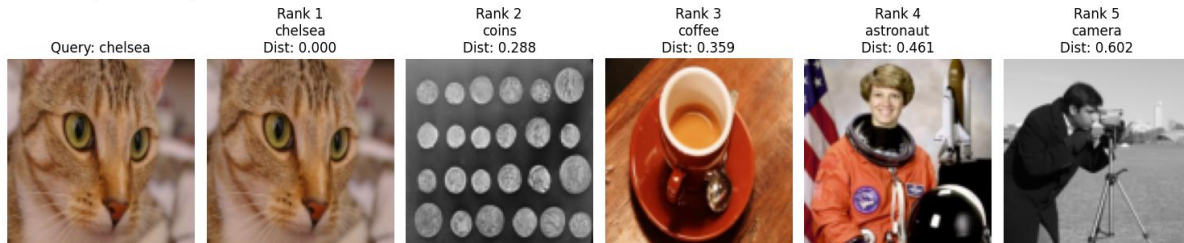
Hasil Praktik

Memproses database citra...

- astronaut diproses.
- camera diproses.
- coffee diproses.
- coins diproses.
- chelsea diproses.

Hasil Retrieval (semakin ke kanan, semakin tidak mirip):

Rank 1: chelsea (Distance: 0.000)
Rank 2: coins (Distance: 0.288)
Rank 3: coffee (Distance: 0.359)
Rank 4: astronaut (Distance: 0.461)
Rank 5: camera (Distance: 0.602)



Amati urutan citra hasil retrieval. Apakah citra yang secara visual mirip dengan query (berdasarkan warna dominan karena kita menggunakan histogram warna) muncul di peringkat atas (jarak terkecil)? Seberapa baik histogram warna sebagai fitur untuk CBIR dalam kasus ini?

E. Hasil Praktik

Lengkapi tabel hasil praktikum berikut:

No.	Nama Praktikum	Hasil Praktikum
1	Menghitung Jarak Berbasis Piksel (L1 & L2)	
2	Menghitung Cosine Similarity (Histogram Warna)	
3	Menghitung Structural Similarity Index (SSIM)	
4	Penerapan Template Matching	
5	Simulasi Content-Based Image Retrieval (CBIR) Sederhana	

F. Penugasan

1. Kumpulkan Laporan Praktikum dari jobsheet ini dalam bentuk Microsoft word sesuai dengan format jobsheet praktikum dan dikumpulkan di web elnino polines. (JANGAN DALAM BENTUK PDF)
2. Kumpulkan luaran kode praktikum dalam bentuk ipynb yang sudah diunggah pada akun github masing-masing. Lampirkan tautan github yang sudah di unggah melalui laman LMS elnino.
3. **Fitur Berbeda untuk CBIR:** Modifikasi Praktikum 5. Alih-alih menggunakan histogram warna, coba gunakan fitur yang lebih sederhana seperti **rata-rata warna** (mean R, mean G, mean B) sebagai vektor fitur 3 dimensi. Hitung kemiripan menggunakan Jarak Euclidean (L₂) antara vektor fitur rata-rata warna ini. Bandingkan hasil retrievalnya

dengan hasil yang menggunakan histogram. Manakah fitur yang lebih baik untuk membedakan citra-citra dalam database kecil tersebut?

4. **Template Matching Invariant:** Cari tahu dari dokumentasi `skimage.feature.match_template` atau sumber lain, apakah metode `match_template` standar invariant terhadap rotasi atau perubahan skala? Jelaskan. (Petunjuk: Coba rotasi atau ubah ukuran template pada Praktikum 4 dan lihat hasilnya).

G. Kesimpulan

Mengukur kemiripan antar citra adalah tugas fundamental dalam pengolahan citra dan pengenalan pola. Berbagai ukuran kemiripan dan jarak telah dikembangkan, mulai dari perbandingan langsung nilai piksel (seperti Jarak Euclidean L_1 , L_2 , SSD), perbandingan fitur yang diekstraksi (seperti Cosine Similarity pada histogram), hingga ukuran yang mencoba meniru persepsi visual manusia seperti SSIM. Setiap ukuran memiliki karakteristik, kelebihan, dan kekurangannya sendiri dalam hal sensitivitas terhadap perubahan pencahayaan, geometri, atau struktur. Pemahaman dan pemilihan ukuran kemiripan yang tepat sangat krusial untuk keberhasilan aplikasi pengenalan pola seperti pencocokan template (template matching) untuk menemukan objek, dan Content-Based Image Retrieval (CBIR) untuk mencari citra serupa dalam database besar berdasarkan konten visualnya.

H. Referensi

- Gonzalez, R. C., & Woods, R. E. (2018). *Digital Image Processing*. Pearson.
- Wang, Z., Bovik, A. C., Sheikh, H. R., & Simoncelli, E. P. (2004). Image quality assessment: from error visibility to structural similarity. *IEEE Transactions on Image Processing*, 13(4), 600–612.
- Rout, N. K., Atulkar, M., & Ahirwal, M. K. (2021). A review on content-based image retrieval system: present trends and future challenges. *International Journal of Computational Vision and Robotics*, 11(5), 461–485.
- Hashemi, N. S., Aghdam, R. B., Ghiasi, A. S. B., & Fatemi, P. (2016). Template Matching Advances and Applications in Image Analysis. *American Scientific Research Journal for Engineering, Technology, and Sciences*, 26(3), 91–108.
- Scikit-image Documentation. <https://www.google.com/search?q=https://scikit-image.org/docs/stable/api/>
- Scipy Spatial Distance Documentation. <https://docs.scipy.org/doc/scipy/reference/spatial.distance.html>